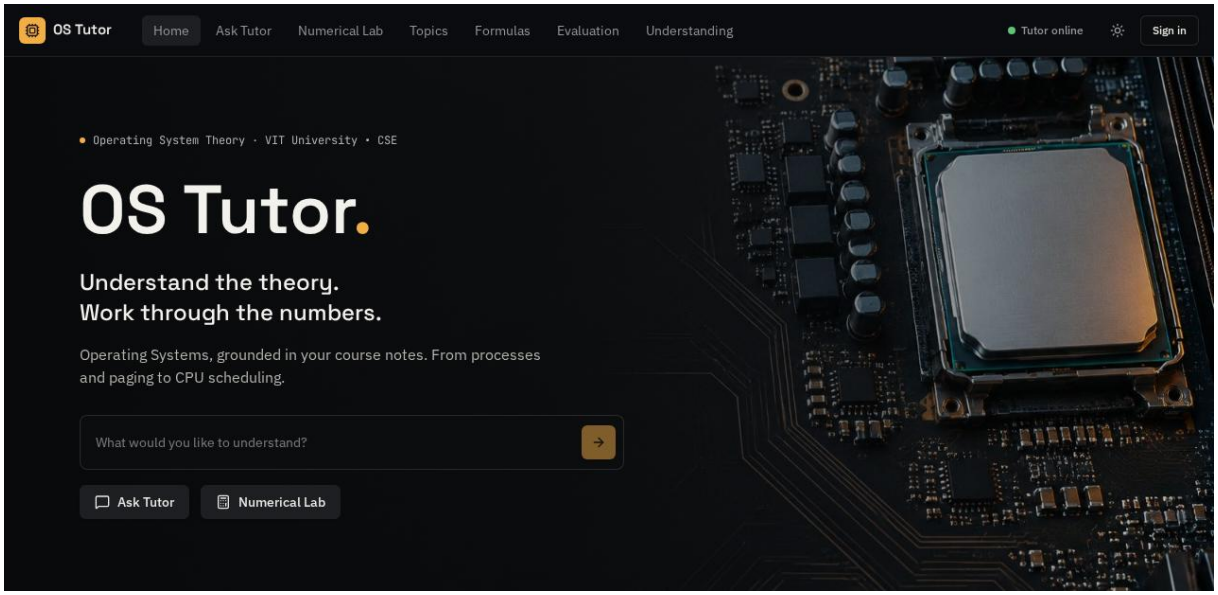
VIT University, Vellore

Operating Systems Lab · Course Project · Final Review

OS Tutor

An LLM-Based Tutor to Teach Operating System Concepts



The live OS Tutor website (home page)

Field	Details
Name	Madhav Tiwary
Registration Number	24BCE2576
Team	Solo Project
Faculty	Jagalingam P
Department / University	CSE, VIT University, Vellore
Academic Year	2026-27
Live website	https://os-tutor.lovable.app
Live tutor API	https://os-tutor-api.onrender.com

Abstract

OS Tutor is a web-based AI tutor that teaches Operating System concepts using a Large Language Model (LLM) grounded in the actual VIT course material through Retrieval-Augmented Generation (RAG). A student can ask a theory question in plain English and receive an answer written only from the seven course modules, with the module and page of every source shown. Unlike a general chatbot, OS Tutor also solves CPU-scheduling numericals (FCFS, SJF, Priority, Round Robin) with exact deterministic solvers and an animated Gantt chart, so numbers are never guessed by the language model. Finally, an Evaluation area writes multiple-choice and true/false questions from the course notes, rewards right answers, teaches the topic after wrong answers, and tracks each student's understanding per topic up to 100%.

This Final Review documents the complete journey: the Kaggle prototype (Review 1), the cleaning and metric work (Review 2), and everything since: the exact numerical solver, the FastAPI backend, the move to free hosting with Groq and Render, GitHub-based deployment, the student website and the evaluation and understanding-tracking system. The project is entirely free to run.

Contents

1. Problem Statement
2. Objectives
3. Background and Literature
4. Project Journey (Review 1 to Final)
5. System Architecture
6. How It Works
7. Features of the Website (with screenshots)
8. Novelty
9. Deployment: Kaggle, GitHub, Render, Groq
10. Hindrances and How They Were Solved
11. Results and Testing
12. Project Structure
13. Limitations and Future Scope
14. Conclusion
References

1. Problem Statement

To develop an LLM-based tutor that teaches Operating System concepts to students, answering their theory questions strictly from the prescribed course material, solving OS numericals exactly, and evaluating and tracking each student's understanding topic by topic.

The Operating Systems laboratory covers many concepts, definitions and algorithms, from processes and threads to paging and CPU scheduling. Students often search back and forth through lecture PDFs to clear a doubt. General-purpose chatbots answer quickly, but they answer from internet knowledge, not from what was taught, and they can "hallucinate" fluent but wrong statements. They are also unreliable at arithmetic: a scheduling numerical needs exact waiting and turnaround times, which a language model can easily get wrong. Finally, a chatbot does not tell a student how well they actually understand each topic.

OS Tutor addresses all three gaps: grounded theory answers, exact numerical solutions, and measurable understanding.

2. Objectives

- Prepare and clean the seven OS course modules as the tutor's only knowledge base.
- Build a RAG pipeline (embeddings + FAISS) that retrieves the right course passages for a question.
- Use an LLM to write answers only from retrieved material, and refuse when the material does not cover the question.
- Add exact, deterministic solvers for CPU-scheduling numericals with Gantt charts.
- Wrap everything in an online API and a polished student website.
- Deploy the whole system for free so any student can use it from a link.
- Add an Evaluation area and an Understanding tracker with student accounts.

3. Background and Literature

Transformer LLMs (Vaswani et al., 2017) produce fluent explanations but can hallucinate (Ji et al., 2023). Retrieval-Augmented Generation (Lewis et al., 2020) grounds answers by first retrieving relevant passages; dense retrieval (Karpukhin et al., 2020) using sentence embeddings (Reimers and Gurevych, 2019) outperforms keyword search. OS Tutor uses BAAI bge-small-en-v1.5 embeddings (Xiao et al., 2023), FAISS similarity search (Johnson et al., 2019) and Maximal Marginal Relevance (Carbonell and Goldstein, 1998) for diverse results. The Kaggle prototype ran Qwen3-8B (Yang et al., 2025) in 4-bit NF4 quantization (Dettmers et al., 2023); the final deployment uses a Qwen3-family model served by Groq.

4. Project Journey (Review 1 to Final)

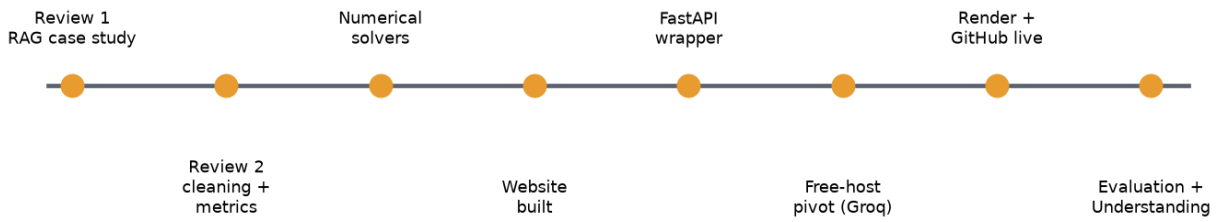


Figure 1: Development timeline from the first review to the final system

Review 1: LLM + RAG case study (Kaggle)

The first version was built in a Kaggle notebook on 2 NVIDIA T4 GPUs. The seven VTOP module PDFs were cleaned into new PDFs, chunked with module and page numbers, embedded with bge-small-en-v1.5 and stored in a FAISS index (chunks.pkl, embeddings.npy, faiss.index). Qwen3-8B was loaded in 4-bit to fit in GPU memory (about 6.6 GiB used) and answered questions from the retrieved chunks. It worked for theory but only inside a notebook.

Review 2: Cleaning, retrieval quality and metrics

Answers were noisy: broken words ("sem pha re" for semaphore), random capitals, one-sided comparisons and stray characters. Review 2 added the junk-chunk filter, per-concept retrieval, MMR, a non-English token block, word repair using a course-term list (TECH_WHITELIST), terminology normalisation and length trimming. Retrieval similarity was introduced as the metric (top match 0.8115 for "process vs thread"). The review listed two next steps: numerical questions and a proper test set.

After Review 2: to the final product

Stage	What was done
Numerical solvers	Exact FCFS, SJF (non-preemptive), Priority (non-preemptive) and Round Robin with quantum; idle gaps; four input formats; refusal for unsupported algorithms; "needs input" when data is missing.
Router	One function decides: theory, numerical, needs_input, unsupported or error. The website shows a different view for each.
Website	Student site with Ask Tutor, Numerical Lab, Topics, Formulas; chat history; dark design.
FastAPI backend	Kaggle code wrapped into an online API (/api/ask, /api/health, later /api/quiz) with the original code kept almost verbatim.
Free hosting pivot	GPU hosting costs money; Hugging Face Spaces now needs a paid plan. Retrieval kept on CPU; answer writing moved to the free Groq API.

GitHub + Render	Code pushed to GitHub; Render Blueprint (render.yaml) builds the Docker image and serves it free.
Evaluation	Course-grounded quiz questions, +10 / -5 scoring, lessons after wrong answers, student accounts and an Understanding page.

5. System Architecture

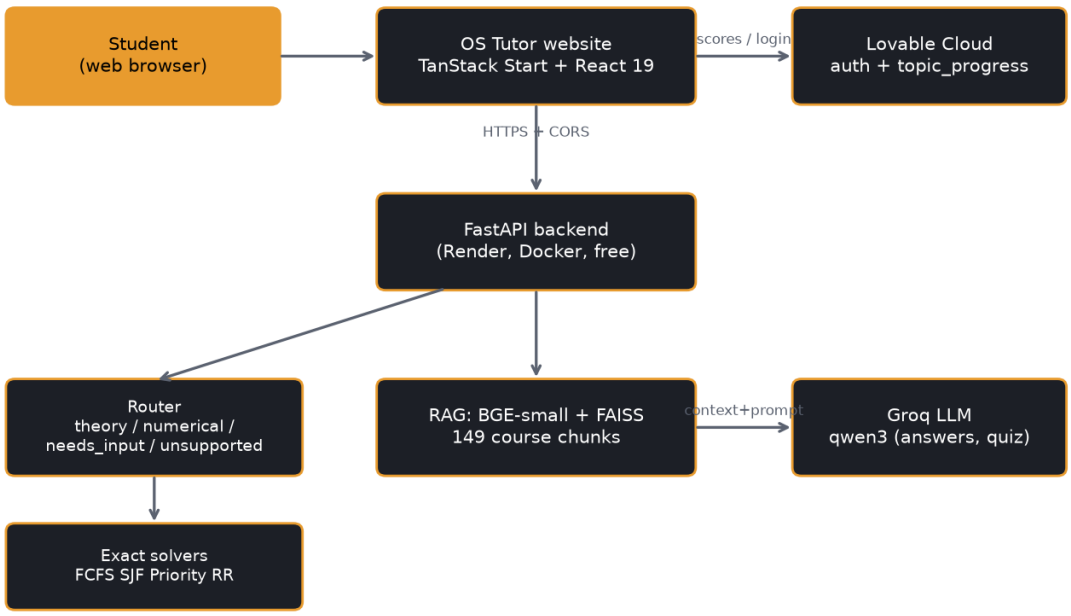


Figure 2: Overall architecture of OS Tutor

The student only ever uses the website. The website talks directly to the FastAPI backend on Render for answers, numericals and quiz questions, and to Lovable Cloud for sign-in and saved understanding scores. Inside the backend, a router sends theory questions through the RAG pipeline to the Groq LLM and numerical questions to the exact solvers. The language model never computes a scheduling result.

Layer	Technology	Role
Frontend	TanStack Start v1, React 19, Tailwind CSS v4	Pages, chat, Gantt chart, quiz, progress bars
Accounts & scores	Lovable Cloud (auth + database)	Email / Google sign-in; topic_progress table; record_answer() function
API	FastAPI + Uvicorn in Docker	/api/ask, /api/quiz, /api/health
Retrieval	FastEmbed BAAI/bge-small-en-v1.5, FAISS (CPU)	149 course chunks, 384-dimension vectors
Generation	Groq API, Qwen3-family model	Writes answers and quiz questions from retrieved text
Cleanup	wordninja, pyspellchecker, TECH_WHITELIST	Same output repair as Review 2
Hosting	Render free web service, GitHub	Builds automatically from the repository

6. How It Works

6.1 Theory questions: the RAG pipeline

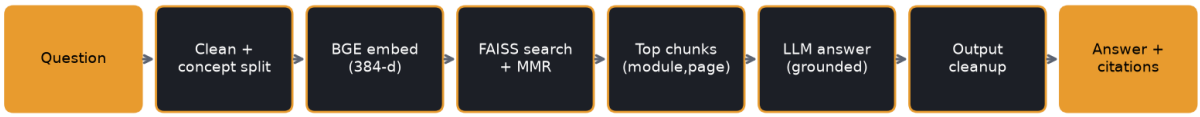


Figure 3: Path of a theory question

The question is cleaned and, if it names several concepts (e.g. "process vs thread"), each concept is searched separately. The question is turned into a 384-number vector, FAISS finds the most similar course chunks, and MMR keeps them varied. Only these chunks are given to the LLM with the rule "answer using only the provided course material". The output goes through the Review 2 cleanup and is shown with its module/page sources. If the material does not cover the question, the tutor says "That information is not available in the provided course material." instead of guessing.

6.2 The router (decision tree)

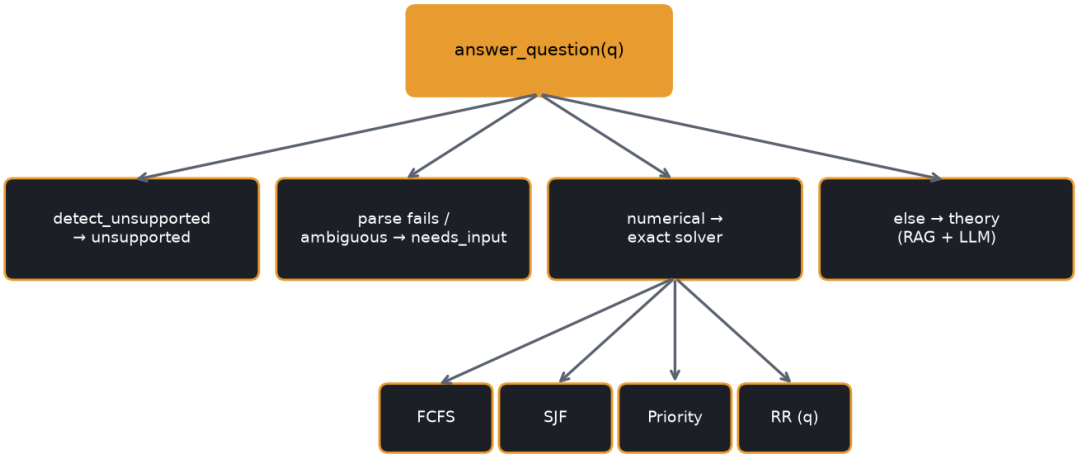


Figure 4: How every question is classified

6.3 Numerical questions: exact solvers

Scheduling numericals are parsed into processes (arrival, burst, priority) and solved by plain Python code, not the AI. The result includes the Gantt chart (with IDLE slices), completion, turnaround and waiting times, and the averages. Formulas used:

```
Turnaround Time (TAT) = Completion Time - Arrival Time
Waiting Time (WT) = Turnaround Time - Burst Time
```

```
Average WT = Sum(WT) / n     Average TAT = Sum(TAT) / n
```

Unsupported algorithms (e.g. SRTF / preemptive) get a clear refusal listing the supported ones, and questions with missing data get a "needs input" reply asking for exactly what is missing.

6.4 Evaluation and understanding tracking

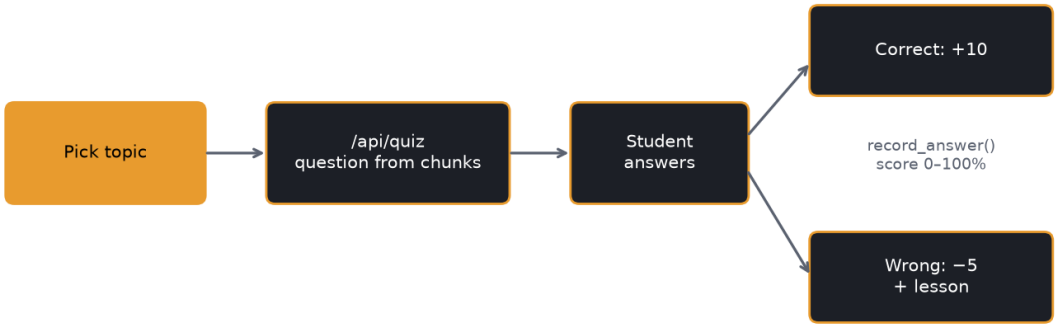


Figure 5: Evaluation learning loop

The student picks one of 15 OS topics. The backend retrieves course chunks on that topic and asks the LLM to write one MCQ or true/false question from them; the question is strictly checked (options, one correct answer, explanation, sources) and discarded if invalid, never patched or invented. A right answer adds 10 points; a wrong answer removes 5, shows the correct answer and immediately teaches the topic using the RAG tutor. Scores (0-100%) are saved to the student's account by a single database function, so the page itself never computes stored scores.

7. Features of the Website

Live at https://os-tutor.lovable.app. All screenshots below are from the running system connected to the real backend.

7.1 Ask Tutor (grounded theory answers with citations)

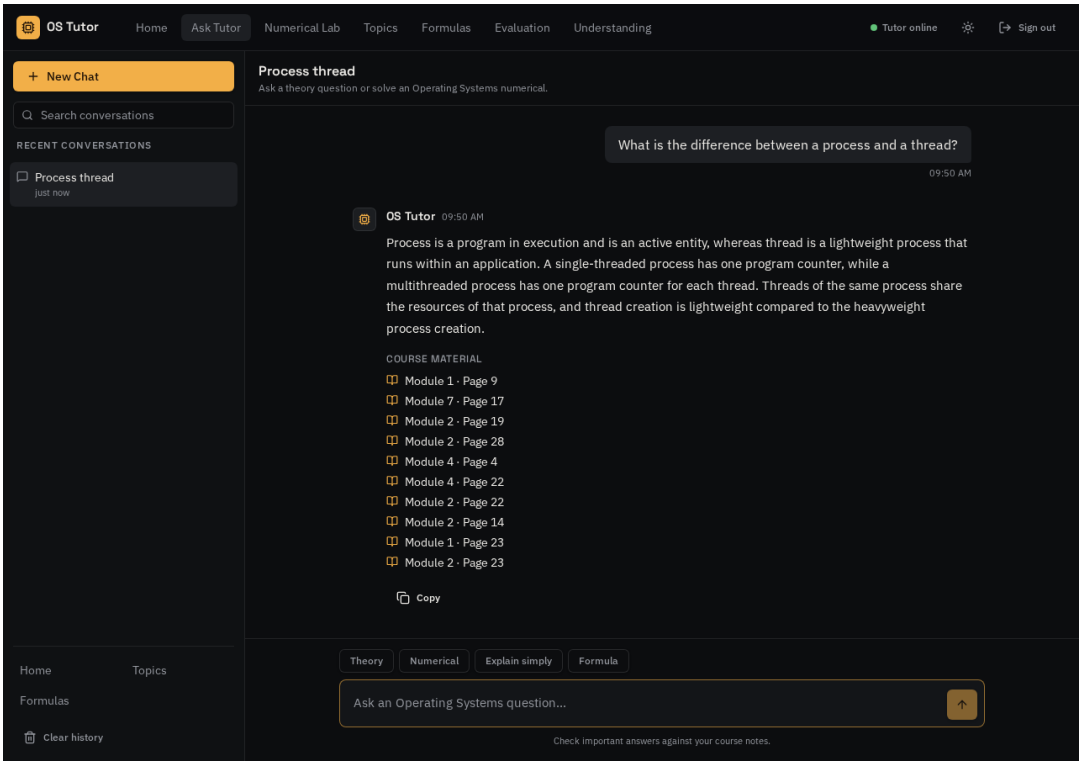


Figure 6: A real answer to "What is the difference between a process and a thread?" with module/page sources

7.2 Numerical Lab (exact CPU scheduling with Gantt chart)

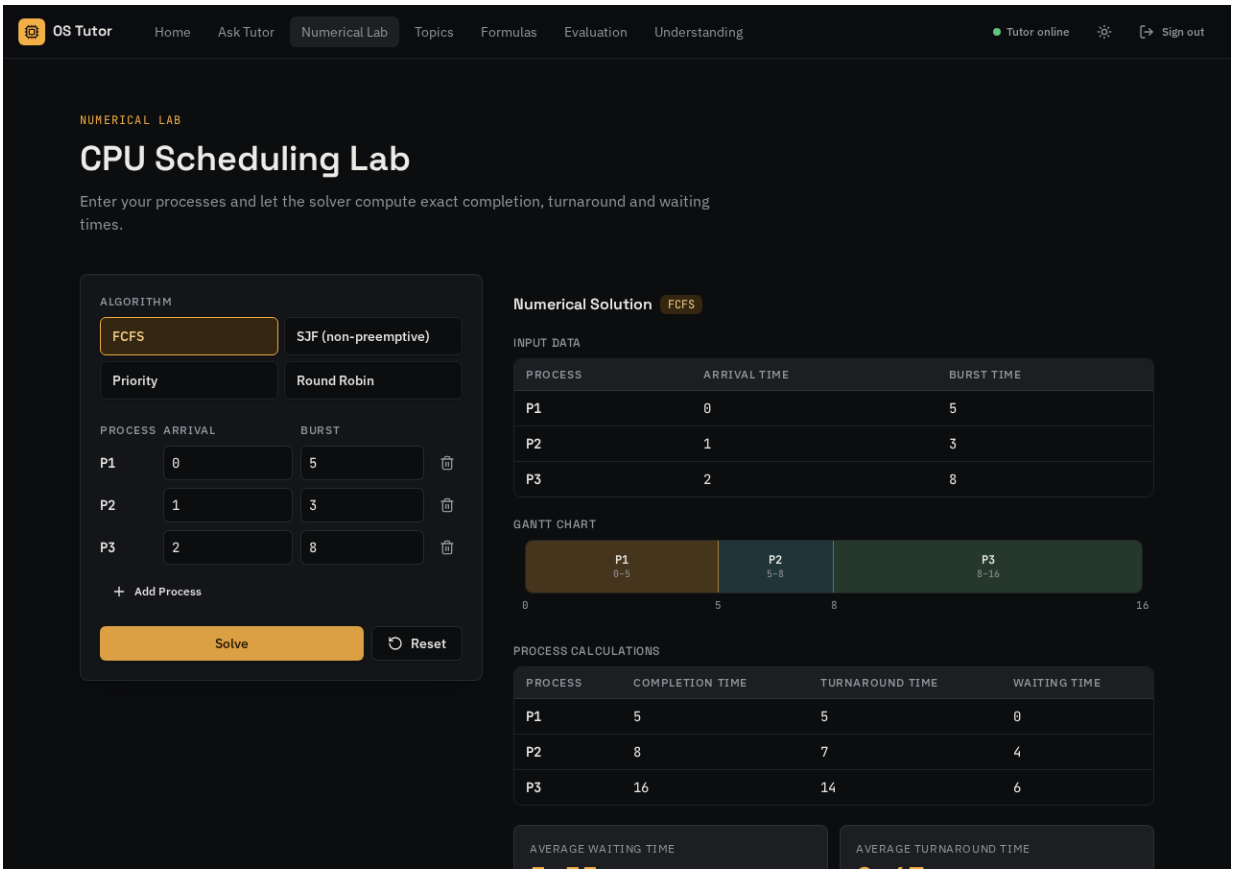


Figure 7: FCFS for P1(0,5), P2(1,3), P3(2,8): Gantt chart and per-process times

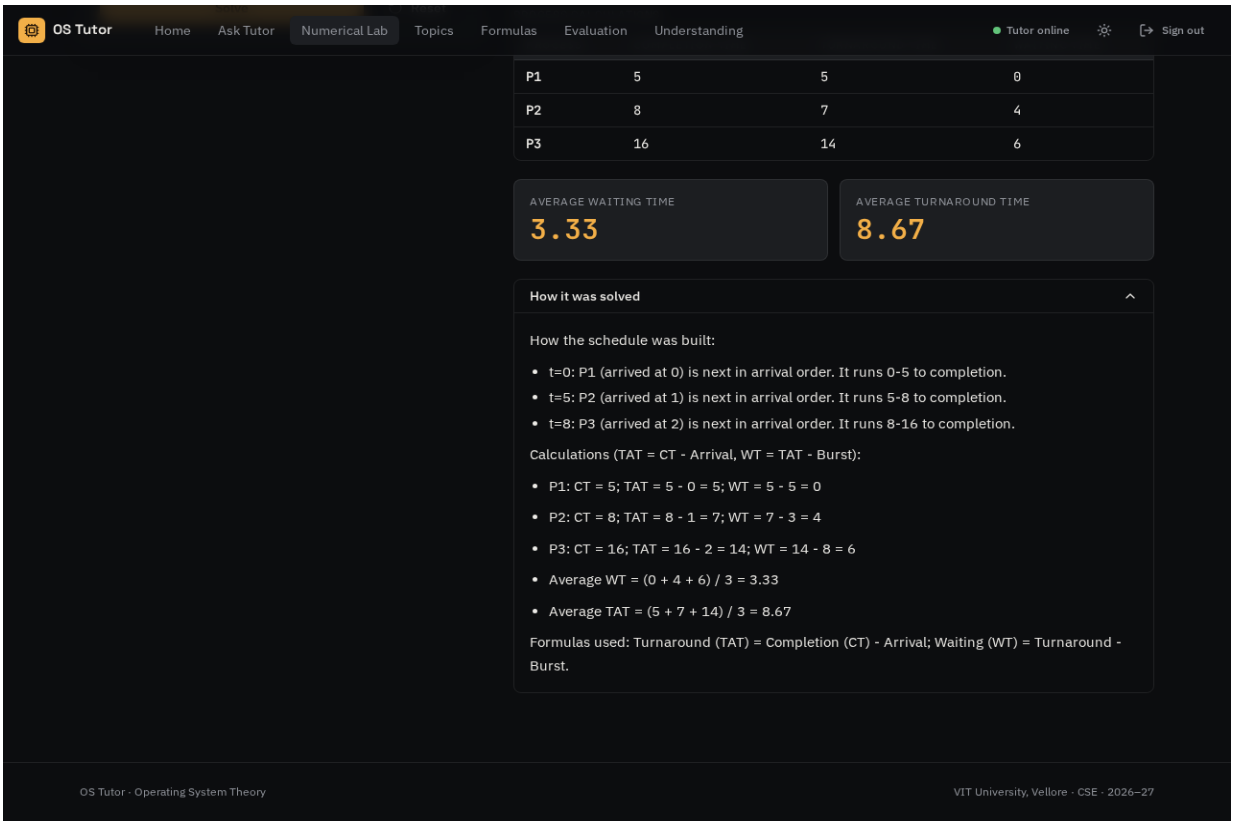


Figure 8: Averages (Avg WT 3.33, Avg TAT 8.67) and the step-by-step working

7.3 Evaluation (course-grounded quiz)

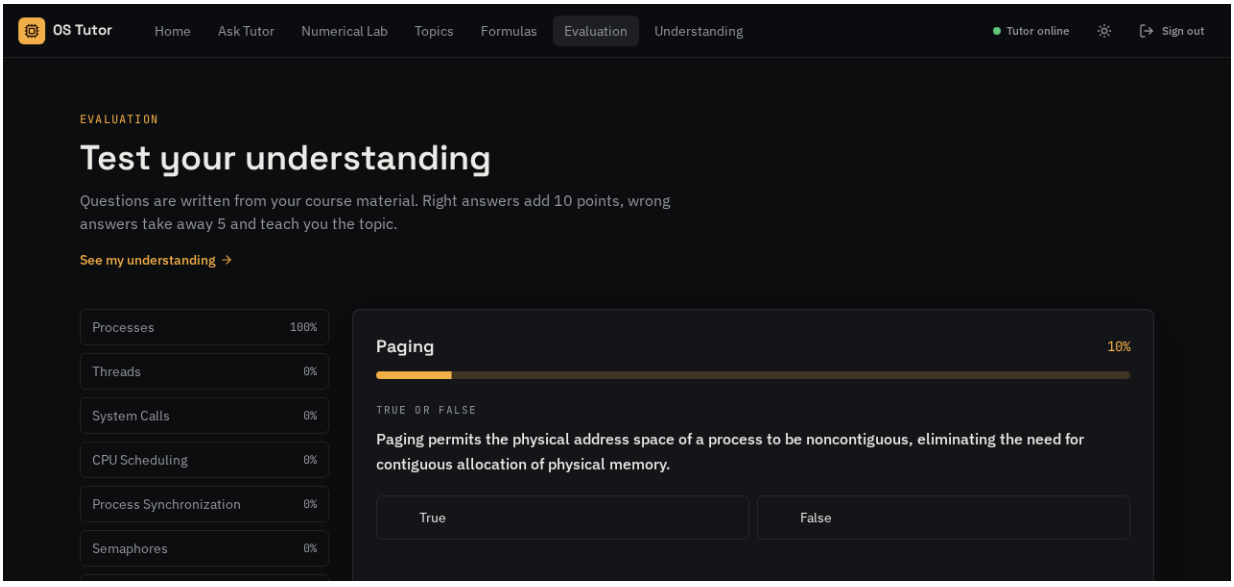


Figure 9: A true/false question on Paging written from the course notes

7.4 Understanding tracker

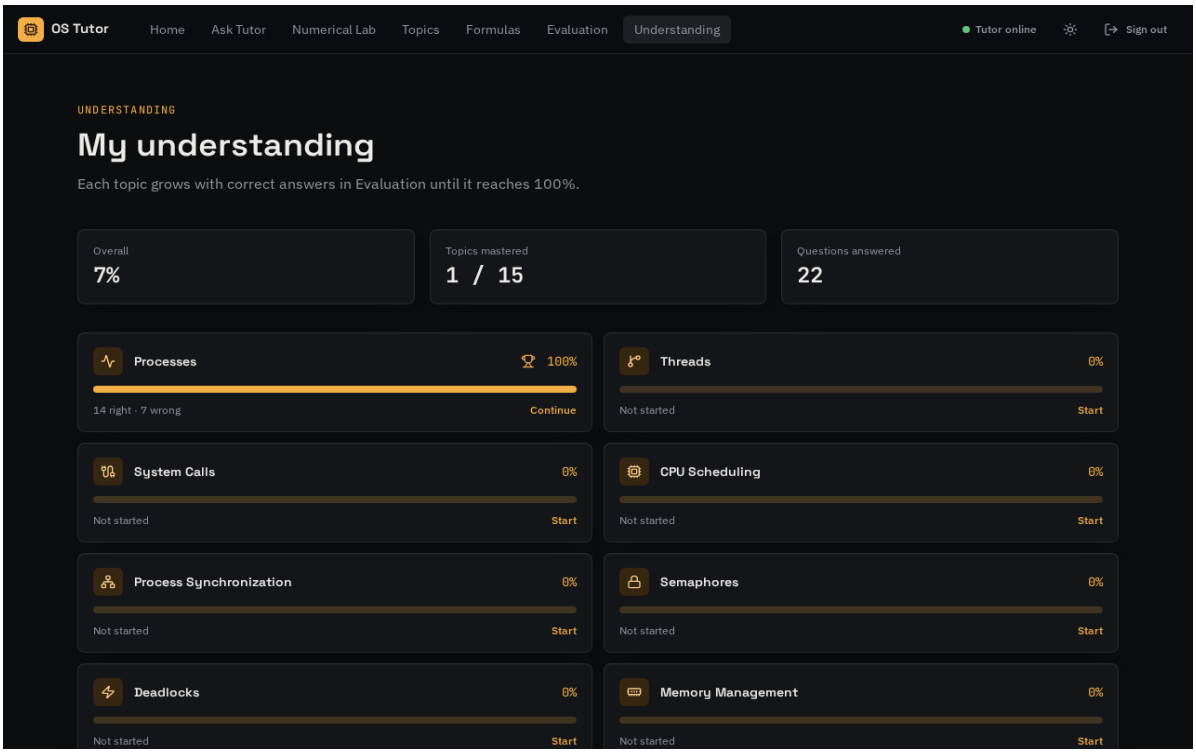


Figure 10: Per-topic understanding, overall %, topics mastered and answers given

7.5 Topics and Formula reference

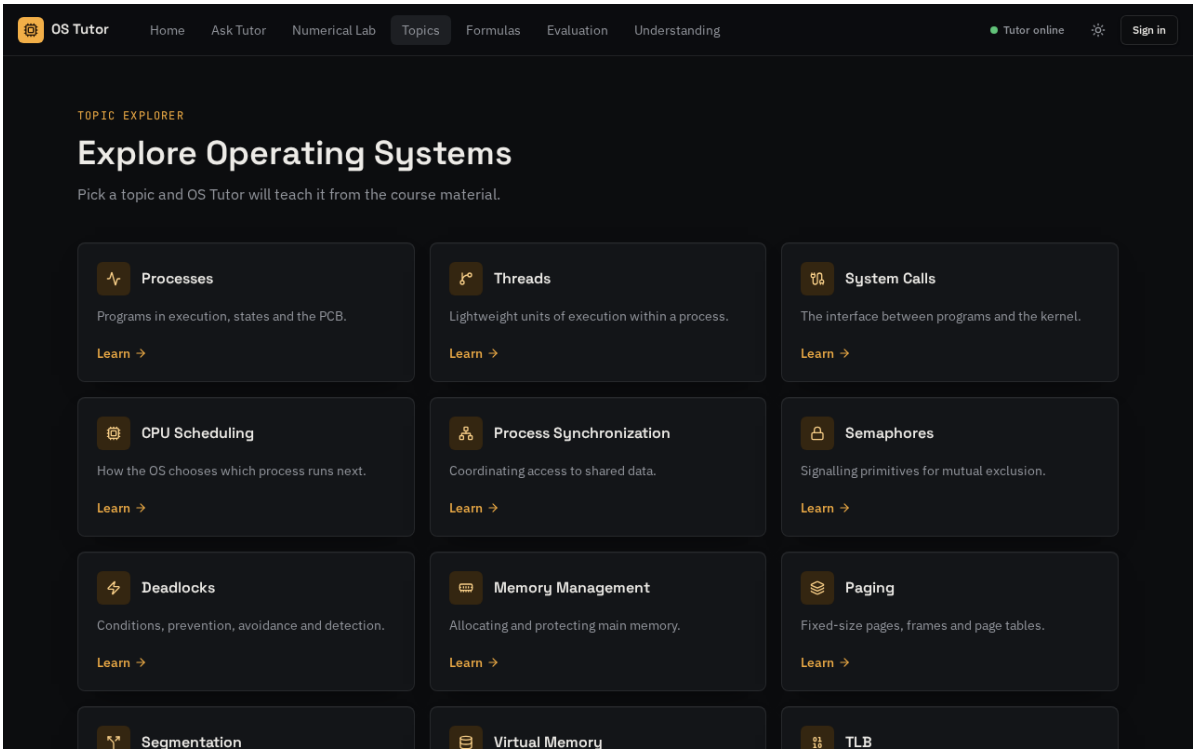


Figure 11: Topics page

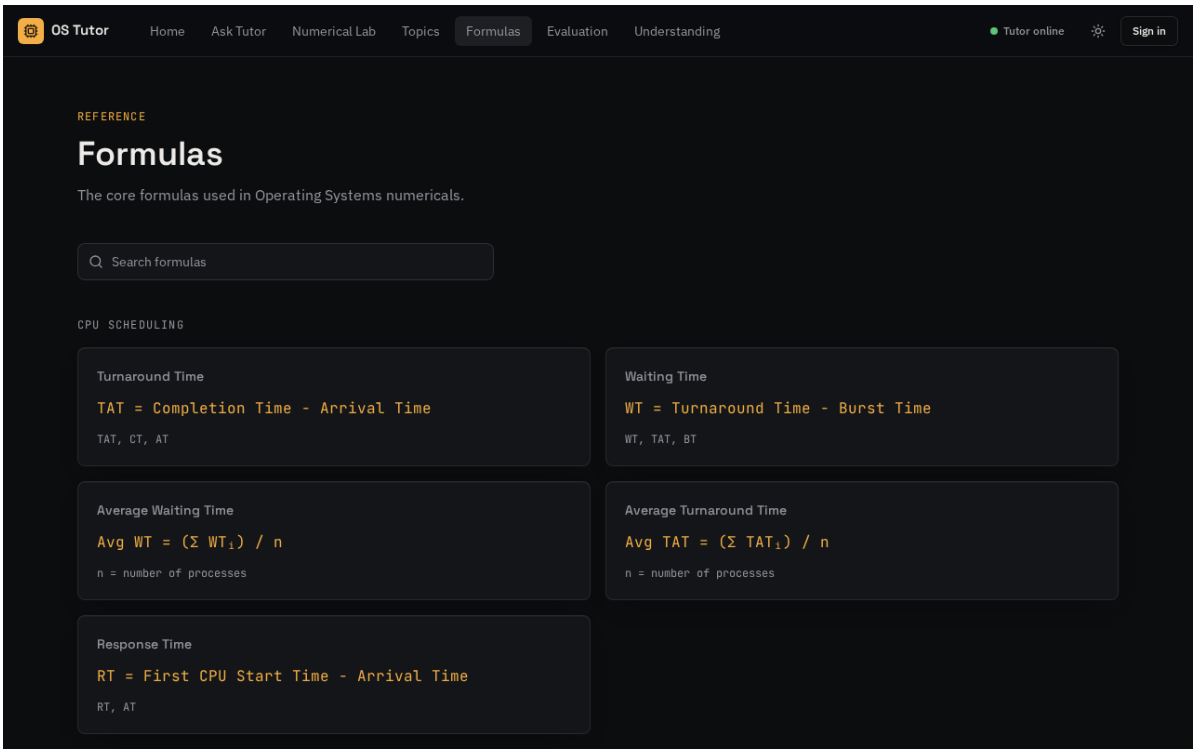


Figure 12: Formula reference page

7.6 Other product features

- Chat history with one link per conversation, search and "clear history".
- "Tutor online / warming up" indicator, since the free server sleeps when idle.
- Friendly messages when the tutor is busy or offline, instead of crashes; every reply from the server is checked before it is shown.
- Light and dark themes, works on phones, sign-in with email or Google.
- No made-up follow-up suggestions: the tutor only offers what it can actually answer.

8. Novelty

Aspect	General chatbot	OS Tutor
Knowledge source	Internet-scale, unknown	Only the 7 VIT OS modules (149 chunks)
Citations	Usually none	Module and page for every answer
Out-of-syllabus questions	Answers anyway	Clearly declines
Numericals	LLM arithmetic, may be wrong	Deterministic exact solver + Gantt chart
Missing data	Assumes values	Asks for the missing input
Assessment	None	Course-grounded quiz, +10/-5, lesson on mistakes
Progress	None	Per-topic understanding saved to account
Cost	Subscription / GPU	Runs entirely on free services

The key idea is a hybrid tutor: the LLM is used only for what it is good at (explaining in natural language from given text, and writing questions from given text), while everything that must be exact (numbers, scores) is handled by deterministic code. Combined with a teach-on-mistake evaluation loop and topic-wise understanding tracking, OS Tutor becomes a complete learning tool rather than a chatbot.

9. Deployment: Kaggle, GitHub, Render, Groq



Figure 13: Deployment pipeline

- Kaggle: research and prototype, GPU notebook, produced chunks.pkl, embeddings.npy and faiss.index.
- Lovable project: the website and the backend folder live together in one project.
- GitHub: the project is synced to a GitHub repository, which is the single source for deployment.
- Render: a Blueprint file (render.yaml) tells Render to build the backend Docker image and run it on the free plan at os-tutor-api.onrender.com. The Groq key is stored as a secret environment variable.

- Groq: provides the LLM through a free API. Limits are per account and can change, so the site shows "the tutor is busy" if rate-limited.
- CORS: the backend only accepts browser requests from the listed website addresses (published site, preview, editor and local testing).

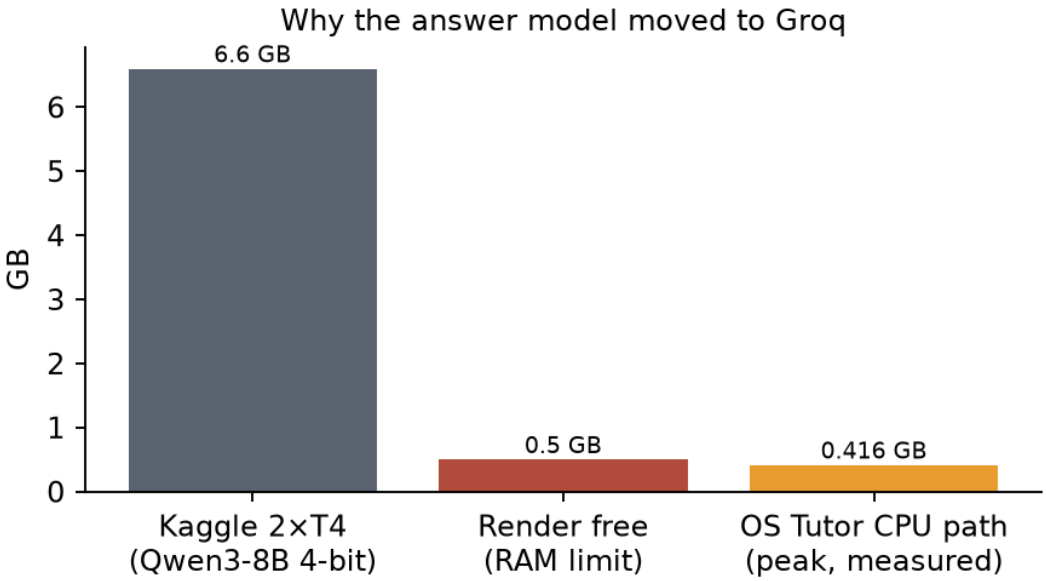


Figure 14: Memory needs vs. free hosting limit

10. Hindrances and How They Were Solved

Problem	Impact	Solution
Noisy VTOP text (Review 2)	Bad characters, broken words	Hand-cleaned module PDFs + output repair
One-sided comparisons	"Process vs thread" only explained one	Per-concept retrieval + prompt rule
LLM unreliable at arithmetic	Wrong waiting/turnaround times	Exact deterministic solvers
No free GPU hosting	8B model cannot run online free	CPU retrieval + Groq API for generation
Hugging Face Spaces became paid	Original deployment plan blocked	Switched to Render free web service
Qwen3-32B removed from Groq free list	Planned model unavailable	Verified and used the currently offered Qwen3 model
512 MB Render memory	Heavy libraries would crash	FastEmbed (no PyTorch), 1 thread; peak about 416 MB
Cloudflare block on plain Python HTTP	Test calls to Groq returned 403	Used a proper HTTP client; verified with curl
CORS "Backend unavailable"	Website could not reach API	Added every site origin to CORS_ORIGINS on Render
Render auto-deploy	Changes not live until pushed	Manual "Deploy latest commit" after GitHub sync
Free server sleeps after idle	First request slow (about 1 min)	"Tutor warming up" status + long timeout
Practice mode unreliable	Broken feature	Removed; replaced by validated Evaluation

11. Results and Testing

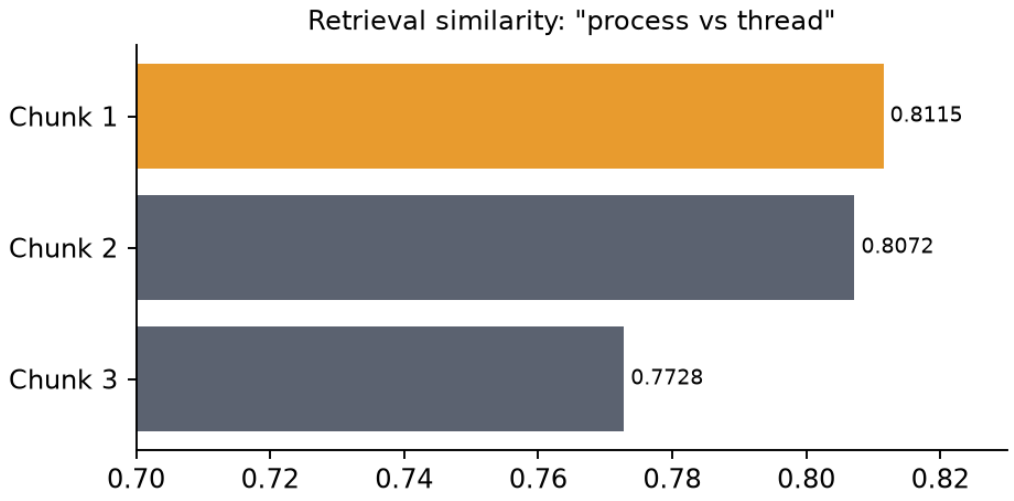


Figure 15: Retrieval similarity of the top 3 chunks (Review 2 metric)

Test	Result
Health check	ok - 149 chunks - FAISS dim 384 - generator groq
Theory: "What is a PCB / process vs thread?"	Grounded answers with Module/Page citations (Fig. 6)
FCFS P1(0,5) P2(1,3) P3(2,8)	Gantt 0-5-8-16; Avg WT 3.33, Avg TAT 8.67 (exact)
SJF, Priority, Round Robin	Exact results with idle slices
SRTF / preemptive question	Refused with supported-algorithm list
Missing data / ambiguous	"Needs input" asking for the missing values
Out-of-material question	"Not available in the provided course material"
Quiz generation (Paging, Threads, Deadlocks, Semaphores)	Valid MCQ / T-F from course chunks with sources
Scoring (signed-in student)	Correct: +10; wrong: -5 + lesson; clamped 0-100
Automated frontend tests	All pass; typecheck clean; build OK

12. Project Structure

```
os-tutor/
|-- render.yaml                 Render Blueprint (free web service)
|-- backend/                    FastAPI tutor (Docker)
|   |-- main.py                 /api/ask /api/quiz /api/health
|   |-- rag.py                  Kaggle RAG code: retrieval, prompt, cleanup
|   |-- numerical.py            FCFS / SJF / Priority / RR exact solvers + router
|   |-- quiz.py                 Course-grounded question writer + validator
|   |-- config.py               GENERATOR (local | groq), CORS origins
|   |-- Dockerfile, requirements-cpu.txt
|   +-- data/                   chunks.pkl embeddings.npy faiss.index
```

```
|-- src/
|   |-- routes/                index, ask, numerical, topics, formulas,
|   |   |-- auth.tsx           sign in (email / Google)
|   |   +-- _authenticated/    evaluate.tsx progress.tsx
|   |-- services/api.ts        single gateway to the backend
|   |-- types/api.ts           validation of every server reply
|   |-- lib/progress.ts        scores via record_answer()
|   +-- components/            chat, Gantt chart, response views, header
+-- drizzle/migrations/        topic_progress table + record_answer()
```

13. Limitations and Future Scope

- The knowledge base is the seven modules; standard textbooks could extend it.
- Free hosting means a cold start after idle and Groq limits that can change.
- A larger scored test set across every module would give a formal accuracy figure.
- Teacher dashboard to view class-wide understanding.

14. Conclusion

OS Tutor grew from a Kaggle notebook into a complete, publicly available learning product. It teaches Operating System concepts using an LLM grounded in the actual course material, solves scheduling numericals exactly, and measures each student's understanding through a teach-on-mistake evaluation loop. The whole system runs on free services, deployed from GitHub to Render with Groq for generation, and is available at https://os-tutor.lovable.app.

References

- Vaswani et al., "Attention Is All You Need", NeurIPS 2017.
- Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS 2020.
- Karpukhin et al., "Dense Passage Retrieval for Open-Domain QA", EMNLP 2020.
- Reimers and Gurevych, "Sentence-BERT", EMNLP 2019.
- Xiao et al., "C-Pack: BGE Embeddings", 2023.
- Johnson et al., "Billion-scale similarity search with GPUs (FAISS)", 2019.
- Carbonell and Goldstein, "MMR for Reordering Documents", SIGIR 1998.
- Ji et al., "Survey of Hallucination in NLG", ACM Computing Surveys 2023.
- Dettmers et al., "QLoRA", 2023.
- Yang et al., "Qwen3 Technical Report", 2025.
- Silberschatz, Galvin, Gagne, "Operating System Concepts".